

python-zero-to-projects

Chapter 1

Introduction to Python

Study Notes

Beginner-friendly notes for absolute beginners - no prior programming experience required

WELCOME TO THE COURSE

This is Chapter 1 of a 21-chapter journey through Python, from absolute basics to real, working projects. Every chapter in this repository follows the same simple structure: **Notes -> Practice Set -> Solutions**. You are currently reading the Notes for Chapter 1. A separate Practice Set PDF accompanies this chapter so you can apply what you learn. Since this is the very first chapter, there are no solutions to review yet - just dive in, follow along, and try things out for yourself.

Section 1. What Is Python, and Why Learn It?

Before installing anything, it helps to understand what you are actually about to learn. This section explains what a programming language is, what makes Python special, and where it is used in the real world.

What Is a Programming Language?

A computer only understands instructions written in very precise, structured ways. A programming language is a set of rules and words that lets a human write instructions that a computer can follow. You write the instructions (called **code**), and the computer reads and carries them out, one step at a time, exactly as written.

What Makes Python Different and Popular?

There are hundreds of programming languages, but Python has become one of the most widely used languages in the world. A few reasons stand out:

- **Readability:** Python code looks close to plain English. A line like `print("Hello")` is easy to read even before you know the rules, which makes Python an excellent first language.
- **Simplicity:** Python avoids a lot of the extra symbols and strict setup that other languages require, so you can focus on learning how to think like a programmer rather than fighting the language itself.
- **Huge community:** Millions of developers use Python, which means there are countless tutorials, forums, and free libraries (pre-written code you can reuse) available whenever you get stuck.

Where Is Python Used in the Real World?

Learning Python opens the door to many different fields. A few of the most common areas are:

- **Web development:** Building the back-end (server-side) logic of websites and web applications.
- **Data science:** Analyzing data, finding patterns, and creating charts and reports from large datasets.
- **Automation:** Writing scripts that perform repetitive tasks automatically, such as renaming files or sending emails.
- **Artificial intelligence and machine learning:** Building systems that learn from data, from recommendation systems to chatbots.

Keep these in mind as motivation: every program you write in this course, no matter how small, is a building block toward one of these areas.

Section 2. Installing Python

To write and run Python code on your own computer, you first need to install Python itself. Follow these steps carefully.

Step-by-Step Installation

- 1 Open your web browser and go to the official Python website: `python.org`
- 2 Hover over (or click) the **Downloads** menu. The site will usually detect your operating system (Windows, macOS, or Linux) and suggest the latest stable version.
- 3 Click the download button to download the installer for your operating system.
- 4 Run the downloaded installer.
- 5 **Windows users:** on the very first installer screen, make sure to check the box that says "**Add python.exe to PATH**" (or "Add Python to PATH") before clicking Install. This step is easy to miss but very important - it lets you run Python from any folder in the terminal.
- 6 **macOS users:** follow the installer prompts normally. macOS may already include an older version of Python, so installing the latest version from `python.org` ensures you are using a current, supported release.
- 7 Finish the installation by following the remaining on-screen prompts.

Verifying the Installation

Once installation finishes, you should confirm that Python is correctly installed and accessible from the terminal (also called the command prompt or shell).

- 1 Open your terminal: Command Prompt or PowerShell on Windows, or Terminal on macOS and Linux.
- 2 Type one of the following commands and press Enter:

```
python --version
# or, on some systems:
python3 --version
```

If the installation worked, you will see output similar to this:

```
> python --version
Python 3.13.1
```

NOTE

If you see an error such as "python is not recognized as an internal or external command," it usually means Python was not added to your system PATH during installation. Re-run the installer and make sure the "Add to PATH" option is checked, or reinstall Python and watch for that step carefully.

Section 3. Setting Up VS Code

Python itself only gives you the ability to run code. To comfortably **write** code, you need a good code editor. This course uses Visual Studio Code (VS Code), a free, lightweight, and extremely popular editor among Python developers.

Installing VS Code

- 1 Go to the official website: `code.visualstudio.com`
- 2 Download the installer for your operating system.
- 3 Run the installer and accept the default options - the defaults work well for almost everyone.
- 4 Open VS Code once installation completes.

Installing the Python Extension

VS Code is a general-purpose editor, so it needs an extension to fully understand Python (things like syntax highlighting, running files, and showing errors).

- 1 Inside VS Code, click the Extensions icon in the left-hand activity bar (it looks like a small set of stacked squares).
- 2 In the search box, type **Python**.
- 3 Find the official extension published by **Microsoft** (it is usually the first result and the most widely installed).
- 4 Click **Install**.

A Quick Tour of the VS Code Interface

Before writing any code, it helps to know the three areas of the interface you will use constantly:

- **File Explorer** (left panel): shows the files and folders in your currently open project, so you can open, create, and organize your Python files.
- **Editor Area** (center/top): the main space where you actually write your Python code.
- **Terminal Panel** (bottom): an integrated command-line where you can run your Python files and see their output, without leaving VS Code.

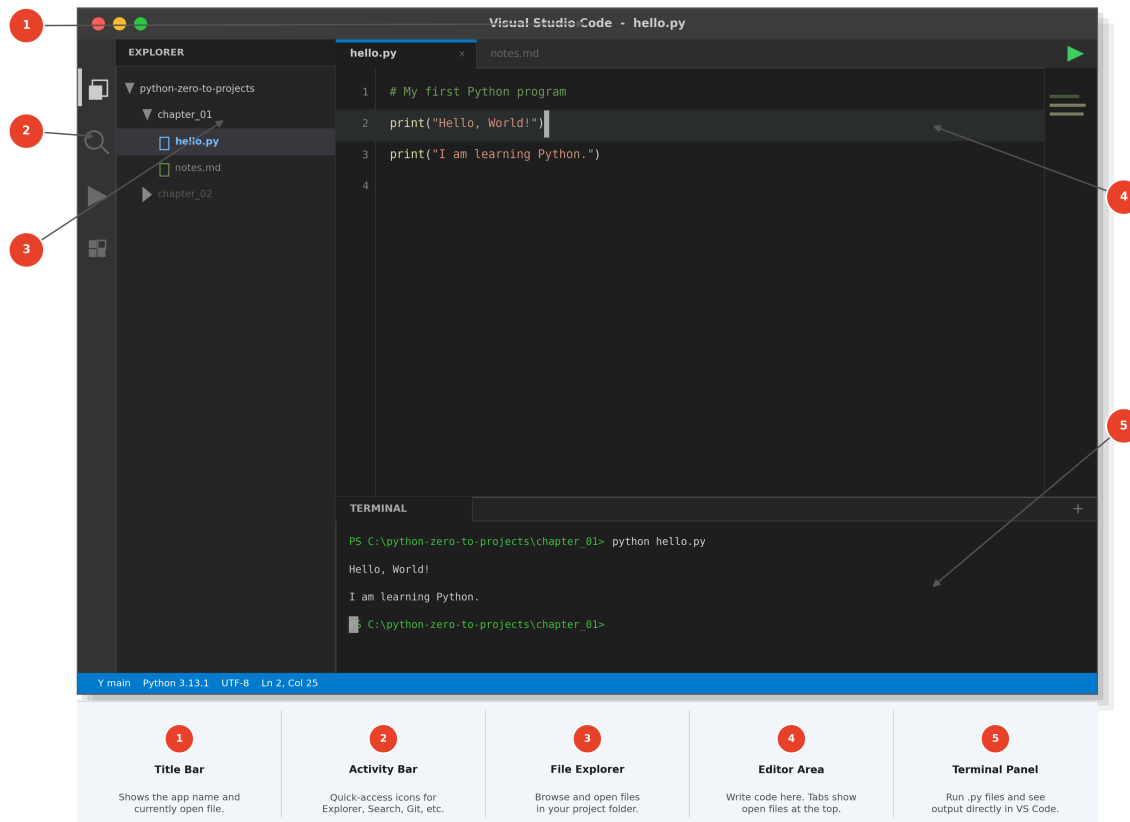


Figure 1. A simplified, labeled layout of the VS Code interface.

Section 4. Running a Python File

Installing Python and VS Code is only half the picture - you also need to know how to actually execute (run) the code you write. This section walks through creating a file and running it.

Creating a .py File

- 1 In VS Code, open the folder where you want to keep your course files (File -> Open Folder).
- 2 In the File Explorer panel, create a new file.
- 3 Name the file with a .py extension, for example hello.py. The .py extension tells your computer and VS Code that this file contains Python code.

Running the File

There are two common ways to run a Python file inside VS Code:

- **Using the Run button:** with your .py file open and active in the editor, click the triangular **Run** (play) button in the top-right corner of the editor. VS Code will automatically open a terminal and run the file for you.

- **Using the integrated terminal directly:** open the terminal panel (Terminal -> New Terminal), make sure you are in the same folder as your file, and type the run command shown below.

```
python filename.py  
# or, on some systems:  
python3 filename.py
```

For example, to run a file named `hello.py`, you would type:

```
> python hello.py
```

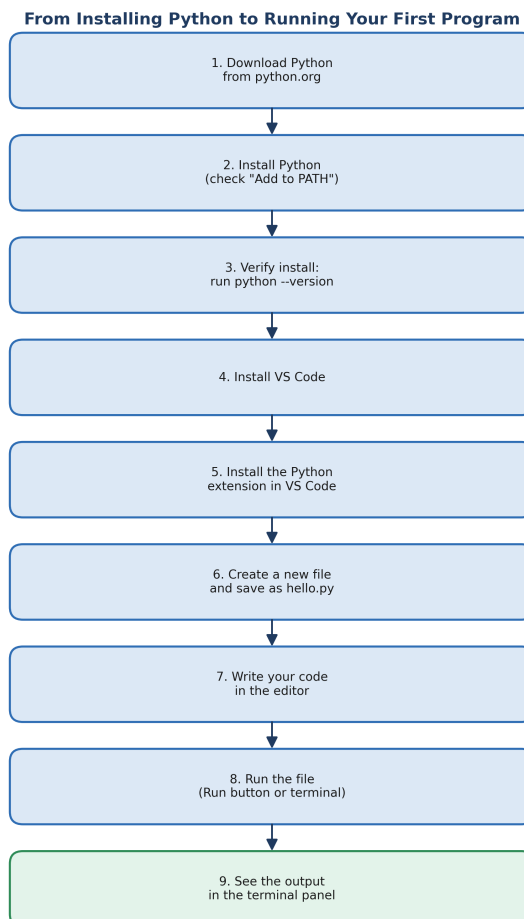


Figure 2. The full journey: from installing Python to seeing your first output.

Section 5. Writing Your Very First Python Program

It is time to write actual code. By tradition, almost every programmer's first program in any language displays the words "Hello, World!" on the screen.

The Classic "Hello, World!" Program

Open your `hello.py` file and type the following single line exactly as shown:

```
print("Hello, World!")
```

Run the file, and you should see this output in the terminal:

```
> python hello.py
Hello, World!
```

Breaking It Down, Line by Line

- `print(...)` is a built-in Python function that displays whatever is inside its parentheses on the screen.
- `"Hello, World!"` is a piece of text, called a **string** in programming. Strings must be wrapped in quotation marks so Python knows it is text rather than code.
- The parentheses () tell Python that you are calling (using) the `print` function and passing it the text inside as input.

Two More Simple Examples

A single line of output is a great start, but let's build a little more confidence with two short additional programs.

Example 1: Printing a Few Lines

```
print("Welcome to Python!")
print("This is my very first program.")
print("I am excited to keep learning.")
```

```
> python multi_line.py
Welcome to Python!
This is my very first program.
I am excited to keep learning.
```

Example 2: Printing a Simple Calculation

Python can also act like a calculator. You can put a math expression directly inside `print()`, and Python will work it out before displaying the result.

```
print(5 + 3)
print(10 - 4)
print(6 * 7)
```

```
> python calculation.py
8
6
42
```

Section 6. Understanding print()

Since `print()` is the very first function you are using, it is worth understanding it a little more deeply.

What print() Does

The `print()` function takes whatever value or values you give it and displays them as text in the terminal/output area. It works with strings, numbers, and the results of calculations.

Printing Strings and Numbers

```
print("I am learning Python")    # prints a string
print(42)                        # prints a number
print(3.14)                      # prints a decimal number
```

```
> python print_basics.py
I am learning Python
42
3.14
```

Printing Multiple Values at Once

You can give `print()` several pieces of information separated by commas. Python will automatically print them on the same line, separated by a single space.

```
print("Score:", 95)
print("Name:", "Alex", "Age:", 16)
```

```
> python print_multiple.py
Score: 95
Name: Alex Age: 16
```

LOOKING AHEAD

The `print()` function actually has more capabilities than shown here, such as choosing a custom separator between values (instead of the default single space) and controlling how the line ends. These details are not essential yet, and will be explored in a later chapter once you are comfortable with the basics.

Section 7. Comments in Python

As your programs grow, you will want to leave yourself (and others) notes inside the code. Python lets you do this with **comments**: text that Python completely ignores when running the program.

Single-Line Comments

A single-line comment starts with a hash symbol `#`. Everything after the `#` on that line is ignored by Python.

```
# This program prints a greeting
print("Hello, World!") # this prints text to the screen
```

Multi-Line (Block) Comments

Python does not have a single, dedicated symbol for multi-line comments the way some other languages do. Instead, the standard approach is to simply stack several single-line `#` comments, one above the other.

```
# This program demonstrates basic printing.
# It was written while following Chapter 1
# of the python-zero-to-projects course.
print("Hello, World!")
```

Why Comments Are Useful

- **Explaining code:** comments can describe what a tricky or important line of code is doing, which helps you (or someone else) understand it later.
- **Leaving notes:** you can leave reminders for yourself, such as things to fix or improve.
- **Temporarily disabling code:** placing a `#` in front of a line of working code turns it into a comment, so Python skips it without you having to delete it. This is useful for testing.

```
print("Hello, World!")
# print("This line will not run, because it is commented out")
```

Section 8. Python Syntax Basics

"Syntax" refers to the set of rules that define how Python code must be written so that Python can understand it correctly. Here are the foundational rules every beginner needs to know from day one.

No Semicolons Required

Many programming languages require a semicolon ; at the end of every line. Python does not. Each line of code simply ends where it ends.

```
# Python style (correct and preferred):  
print("Hello, World!")  
  
# Also technically works, but unnecessary in Python:  
print("Hello, World!");
```

Python Is Case-Sensitive

Python treats uppercase and lowercase letters as completely different characters. This means `print` and `Print` are not the same thing to Python.

```
print("This works correctly")  
  
Print("This causes an error, because Print (capital P) does not exist")
```

Indentation: A Defining Feature of Python

This is one of the most important rules in all of Python. Many programming languages use curly braces { } to group lines of code together into a block. Python uses **indentation** (spaces at the start of a line) instead.

The standard convention, used throughout this course, is **4 spaces** per indentation level. You will see indentation used properly once you reach blocks such as conditionals (if statements) and loops in later chapters, but it is important to recognize the rule right away, since incorrect indentation causes errors even in small snippets of code.

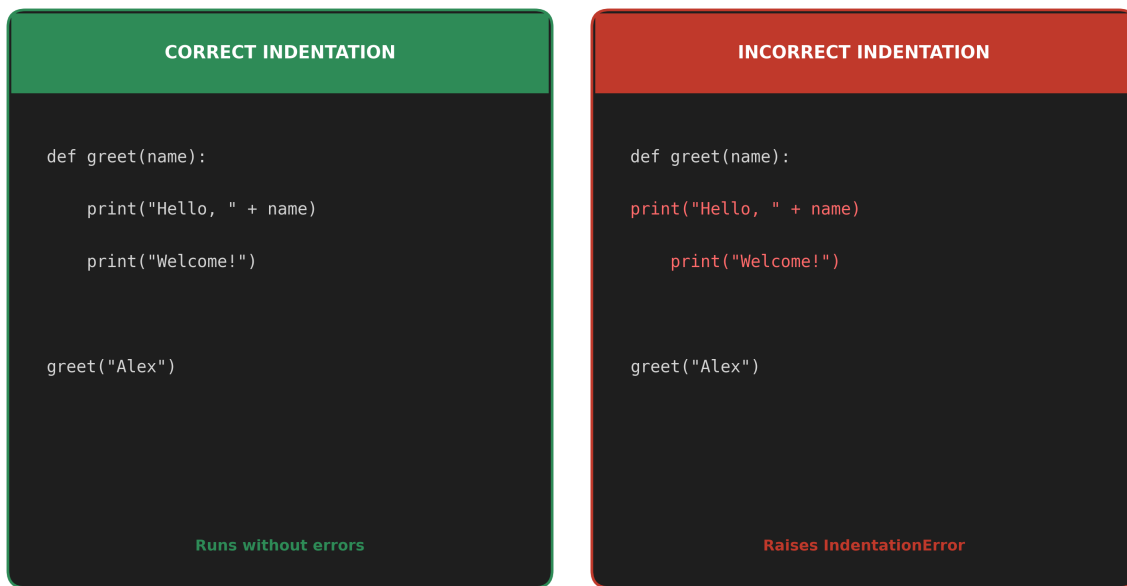


Figure 3. Correct indentation (left) runs fine. Inconsistent indentation (right) raises an `IndentationError`, because the second line is expected to be indented the same amount as the first indented line beneath the function definition.

NOTE

Full conditional statements (if/else) and loops, where indentation really comes to life, are covered in later chapters. For now, simply remember: **consistent indentation matters in Python**, and mixing it up will cause your program to fail with an error.

Common Beginner Mistakes

Almost every Python programmer makes these same mistakes when starting out. Knowing about them in advance will save you time and frustration.

Forgetting quotation marks around text

`print(Hello)` instead of `print("Hello")` - Python will think `He l l o` is the name of something in your code (like a variable) rather than plain text, and will raise an error because it does not recognize that name.

Incorrect indentation

Adding or removing spaces inconsistently causes an `IndentationError`. Python relies on indentation to understand the structure of your code, so it must be applied consistently.

Case-sensitivity mistakes

Typing `Print("Hello")` instead of `print("Hello")`. Since Python is case-sensitive, `Print` with a capital P does not exist, and Python will raise a `NameError`.

Forgetting closing parentheses or quotation marks

Writing `print("Hello)` or `print("Hello"` (missing the closing quote or closing parenthesis) confuses Python about where your text or function call actually ends, resulting in a `SyntaxError`.

Running the file from the wrong folder

Typing `python hello.py` in the terminal while the terminal is pointed at a different folder than the one containing `hello.py` will give you a "file not found" type error. Always check that your terminal is open in the same folder as the file you want to run.

Summary and Recap

Here is a quick recap of everything covered in this chapter:

- Python is a popular, readable, beginner-friendly programming language used in web development, data science, automation, and AI/ML.
- Python is installed from `python.org`, and the installation can be verified with `python --version` or `python3 --version`.
- VS Code, together with its official Python extension, is the editor used throughout this course. Its three core areas are the File Explorer, the Editor Area, and the Terminal Panel.
- A Python file is saved with a `.py` extension and can be run using the Run button in VS Code or with `python filename.py` in the terminal.
- The `print()` function displays text and numbers on the screen, and can display multiple comma-separated values on a single line.
- Comments, written with `#`, are ignored by Python and are used to explain code, leave notes, or temporarily disable a line.
- Python does not require semicolons, is case-sensitive, and uses indentation (commonly 4 spaces) instead of curly braces to define blocks of code.

PRACTICE TIME

Now that you have read through the Chapter 1 notes, open the accompanying **Chapter 1 - Practice Set** PDF and work through all of the coding questions on your own, in your own `.py` files. There are no solutions provided for this first chapter - the goal is simply to get comfortable writing and running code.

Coming Up Next: Chapter 2

Chapter 2 - Variables and Data Types builds directly on what you learned here. You already know how to use `print()` to display fixed text and numbers, and you understand basic Python syntax. In Chapter 2, you will learn how to **store** values using variables, so you can reuse, update, and work with data instead of typing it out every single time.